

SignalR nədir?

Real-time kommunikasiya, WebSocket, server push və ECafe layihəsində
real-time logout mexanizmi

Bu sənəd SignalR-i sıfırdan başa düşmək üçün hazırlanıb. Məqsəd yalnız kod yazmaq deyil, texnologiyanın niyə yarandığını, hansı problemi həll etdiyini və ECafe kimi restoran idarəetmə sistemində necə təhlükəsiz tətbiq oluna biləcəyini izah etməkdir.

Mövzu	Qısa izah
SignalR	ASP.NET Core üçün real-time server-client kommunikasiya kitabxanası.
Əsas istifadə	Server hadisə baş verən kimi browser/app tərəfinə mesaj göndərir.
ECafe nümunəsi	Admin user-i deaktiv edən kimi həmin user-in bütün aktiv ekranları logout edilir.

1. Problem nə idi?

Klassik web sistemlərində browser serverə sorğu göndərir, server cavab verir və əlaqə bitir. Bu model REST API üçün çox yaxşıdır: login et, restoranları çək, müqaviləni yarat, menyunu gətir və s. Amma bəzi hallarda problem yaranır: serverdə hadisə baş verir, browser isə bu hadisədən xəbərsiz qalır.

- Admin user-i deaktiv edir, amma həmin user-in browserində köhnə access token hələ 10 dəqiqə işləyə bilər.
- Mətbəxdə sifariş statusu dəyişir, ofisiant ekranı bunu refresh etmədən görmür.
- Müqavilə owner tərəfindən təsdiqlənir, admin panel bunu dərhal bilmir.
- Stok kritik həddə düşür, mətbəx ekranına dərhal xəbərdarlıq getmir.

Bu problemləri köhnə üsulla həll etmək üçün frontend hər 3-5 saniyədən bir serverə sorğu ata bilər. Bu üsula polling deyilir. Polling işləyir, amma resurs israf edir və gecikmə yaradır.

SignalR bu problemi daha təmiz həll edir: server özü lazımı anda client-ə mesaj göndərir.

2. SignalR nədir?

SignalR Microsoft-un ASP.NET Core üçün hazırladığı real-time kommunikasiya kitabxanasıdır. Sadəcə dillə desək, backend ilə frontend arasında canlı kanal yaradır. Bu kanal açıq qaldığı müddətdə server istədiyi anda frontend-ə hadisə göndərə bilər.

REST API-də front 'məndə yeni məlumat varmı?' deyər soruşur. SignalR-də isə backend 'səndə yeni hadisə var' deyib özü mesaj göndərir.

Model	Kim başlayır?	Nə üçün uyğundur?
REST API	Frontend sorğu göndərir	CRUD, list, detail, create/update/delete əməliyyatları
SignalR	Backend hadisə göndərir	Canlı bildiriş, logout, status update, chat, tracking

3. SignalR niyə yaradılıb?

Web tətbiqlərində real-time ehtiyac artdıqca developer-lər müxtəlif yollar istifadə edirdi: polling, long polling, Server-Sent Events və WebSocket. Hər birinin fərqli browser dəstəyi, connection davranışı və fallback problemi var idi. SignalR bu mürəkkəbliyi developer-dən gizlədir.

- Əgər WebSocket mümkündürsə, SignalR WebSocket istifadə edir.
- WebSocket mümkün deyilsə, fallback olaraq Server-Sent Events və ya Long Polling seçə bilər.
- Connection qırılsa, reconnect mexanizmini idarə edə bilər.
- Client-ləri user, group və connection səviyyəsində hədəfləməyə imkan verir.

Yəni SignalR-in əsas dəyəri təkəcə WebSocket açmaq deyil. Əsas dəyər real-time əlaqəni ASP.NET Core ekosistemində rahat, təhlükəsiz və idarə oluna bilən formaya salmasıdır.

4. WebSocket, SSE, Long Polling fərqi

Texnologiya	Necə işləyir?	Üstünlük	Məhdudiyyət
WebSocket	Client və server arasında daimi iki istiqamətli kanal açılır.	Ən real-time və effektiv modeldir.	Proxy/firewall bəzi mühitlərdə problem yarada bilər.
Server-Sent Events	Server client-ə tək istiqamətli event göndərir.	Bildiriş üçün sadədir.	Client serverə həmin kanal üzərindən mesaj göndərə bilmir.
Long Polling	Client sorğu göndərir, server cavabı hadisə olana qədər saxlayır.	Fallback üçün işləkdir.	Daha çox request və overhead yaradır.
SignalR	Bu texnologiyalardan ən uyğunu avtomatik seçir.	Developer sadə Hub API ilə işləyir.	Əlavə infrastruktur və auth qaydası düşünəlməlidir.

5. SignalR-in əsas anlayışları

SignalR-i başa düşmək üçün bir neçə əsas termin kifayətdir.

Hub

Hub backend-də yaradılan real-time endpoint kimidir. REST controller necə HTTP request qəbul edirsə, Hub da real-time connection qəbul edir. Frontend Hub-a qoşulur və event dinləyir.

ConnectionId

Hər browser tab və ya app connection üçün unikal id yaranır. Bir istifadəçi eyni anda iki tab açsa, iki fərqli connection ola bilər.

Group

Connection-ları qruplaşdırmaq üçündür. Məsələn `userId=15` üçün bütün connection-ları ``user:15`` qrupuna əlavə edirik. Sonra həmin user-in bütün tablalarına eyni mesajı göndərmək olur.

Client event

Backend ``forceLogout`` adlı event göndərir, frontend isə ``connection.on('forceLogout', ...)`` ilə onu dinləyir.

6. ECafe-də real-time logout niyə lazımdır?

ECafe-də rollar və permission-lar biznes baxımından həssasdır. Owner, manager, waiter, kitchen və admin fərqli səhifələr və action-lar görür. Admin bir user-i deaktiv edəndə həmin user-in sistemi istifadə etməyə davam etməsi təhlükəsizlik və biznes riski yaradır.

- Deaktiv user yeni refresh token almamalıdır.
- Mövcud browser session-ları dərhal login səhifəsinə yönləndirilməlidir.
- Backend hər halda həmin user-in request-lərini bloklamalıdır.
- Frontend sadəcə rahat UX üçün dərhal logout edir, əsas təhlükəsizlik backend-də qalır.

Vacib prinsip: SignalR security-nin özü deyil. SignalR istifadəçiyə dərhal xəbər verir. Əsas security refresh token revoke, qısa access token və backend authorization yoxlamasıdır.

7. Real-time logout flow-u

Addım	Nə baş verir?
1	User login olur və access token alır.
2	Frontend <code>`/hubs/auth`</code> Hub-a qoşulur və token göndərir.
3	Backend token-dən <code>userId</code> oxuyur və connection-u <code>`user:{id}`</code> qrupuna əlavə edir.

4	Admin həmin user-i deaktiv edir.
5	Backend user-in refresh tokenlərini revoke edir.
6	Backend `user:{id}` qrupuna `forceLogout` event göndərir.
7	Frontend event-i alır, tokenləri silir və login səhifəsinə yönləndirir.
8	Əgər event çatmasa belə, növbəti API request-də 401/403 gəlir və frontend logout edir.

Bu flow həm real-time UX verir, həm də fallback saxlayır. Browser offline idisə, SignalR event-i almaya bilər. Ona görə backend token və aktivlik yoxlamaları yenə vacibdir.

8. Backend arxitekturası necə olmalıdır?

SignalR kodunu biznes servislərinin içinə dağınıq formada yazmaq düzgün deyil. Clean code üçün real-time bildiriş göndərmə məntiqi ayrıca abstraction altında olmalıdır.

Layer	Məsuliyyət
Api	Hub endpoint-i map edir, JWT auth tətbiq edir.
Application	User deaktiv olanda `IRealtimeNotificationService` çağırır.
Infrastructure/API	SignalR HubContext ilə real event göndərir.
Frontend	Hub-a qoşulur və `forceLogout` event-ini dinləyir.

Sadə backend nümunəsi:

```
public class AuthHub : Hub
{
    public override async Task OnConnectedAsync()
    {
        var userId = Context.User?.FindFirst("userId")?.Value;
        if (!string.IsNullOrEmpty(userId))
            await Groups.AddToGroupAsync(Context.ConnectionId, $"user:{userId}");

        await base.OnConnectedAsync();
    }
}
```

9. Frontend arxitekturası necə olmalıdır?

Frontend-də SignalR connection auth state ilə birlikdə idarə olunmalıdır. User login deyilsə connection açılmamalıdır. Logout olanda connection dayandırılmalıdır. Token dəyişəndə connection yenidən qurulmalıdır.

```
const connection = new HubConnectionBuilder()
    .withUrl(`${API_BASE_URL}/hubs/auth`, {
        accessTokenFactory: () => authStore.accessToken,
    })
    .withAutomaticReconnect()
    .build();

connection.on("forceLogout", (payload) => {
    authStore.logout();
    router.navigate("/login");
});
```

Frontend-də `forceLogout` gəlməsə belə, bütün API response interceptor-larında 401/403 gəldikdə logout məntiqi qalmalıdır. Bu production üçün vacib fallback-dir.

10. Security baxımından nəyə fikir vermək lazımdır?

- Hub endpoint mütləq JWT ilə qorunmalıdır. Anonymous connection qəbul edilməməlidir.
- SignalR query string ilə token göndərə bilər. Buna görə access token qısaömürlü olmalıdır.

- Refresh token deactivate zamanı revoke edilməlidir.
- Backend hər request-də user aktivliyini və permission-u yoxlamalıdır.
- CORS yalnız icazəli frontend origin-lər üçün açıq olmalıdır.
- Hub event adları və payload-lar minimal olmalıdır. Lazımsız şəxsi məlumat göndərilməməlidir.
- Admin action-ları audit log-a yazılmalıdır: kim deaktiv etdi, kimi deaktiv etdi, nə vaxt.
- Production-da çox server varsa SignalR backplane lazımdır: Redis və ya Azure SignalR Service.

11. Tək server və çox server fərqi

Local və sadə production mühitində API bir instance olaraq işləyə bilər. Bu halda SignalR connection-lar həmin instance daxilində saxlanılır. Amma load balancer arxasında 2-3 API instance varsa problem yaranır: user A 1-ci serverə qoşula bilər, admin action-u isə 2-ci serverdə işləyə bilər. 2-ci server 1-ci serverdəki connection-u tanımır.

Bunun həlli SignalR backplane-dir. Redis backplane və ya Azure SignalR Service bütün serverlər arasında real-time mesajları paylaşır.

Mühit	Tövsiyə
Local/MVP	Sadə SignalR kifayətdir.
1 API instance production	Sadə SignalR işləyir, amma monitoring lazımdır.
Çox instance production	Redis backplane və ya Azure SignalR Service istifadə et.

12. SignalR-i harada istifadə etmək olar?

ECafe flow	SignalR dəyəri
Real-time logout	Deaktiv user dərhal sistemdən çıxarılır.
Mətbəx sifarişləri	Yeni sifariş mətbəx ekranına refresh olmadan düşür.
Ofisiant ekranı	Masa/session statusu dərhal yenilənir.
Stok xəbərdarlığı	Azalan stok mətbəx və owner ekranına dərhal göndərilir.
Müqavilə flow-u	Owner təsdiqlədi, admin panel canlı yenilənir.
Notification bell	Yeni app daxili bildiriş gələn kimi icon count artır.

SignalR-i hər şey üçün istifadə etmək lazım deyil. CRUD, siyahı, filter, pagination yenə REST API ilə qalmalıdır. SignalR yalnız hadisə baş verəndə client-ə xəbər vermək üçündür.

13. Best practice tətbiq planı

ECafe üçün tövsiyə olunan mərhələli tətbiq planı:

Mərhələ	İş
1	Backend-də `AuthHub` yarat və JWT ilə qorunmuş `/hubs/auth` endpoint-i əlavə et.
2	`IRealtimeNotificationService` abstraction yarat, service-lər birbaşa HubContext bilməsin.
3	Deactivate staff/user action-larında refresh token revoke + `forceLogout` event göndər.
4	Frontend-də auth lifecycle ilə SignalR connection yarat və logout zamanı dayandır.
5	API interceptor-da 401/403 fallback logout saxla.
6	Production-da çox instance olacaqsa Redis backplane əlavə et.

14. Nəticə

SignalR real-time kommunikasiya üçündür. O, REST API-ni əvəz etmir, onu tamamlayır. ECafe-də SignalR-in ən güclü başlanğıc istifadəsi real-time logout-dur: admin user-i deaktiv edir və həmin user bütün açıq tablardan dərhal çıxarılır.

Ən düzgün yanaşma: backend-də refresh token revoke, qısaömürlü access token, 401/403 fallback logout və SignalR forceLogout birlikdə işləməlidir. Bu kombinasiya həm təhlükəsiz, həm də istifadəçi üçün anlaşıqlı real-time davranış yaradır.